

Sarah Benavides

CS 362 Lab 13

4/16/2026

Linux Forensics (Artifacts Analysis)

Part 1: Non-Technical summary

This lab concentrated on how different tools can be used to investigate and analyze a Linux system from a digital forensics point of view. We began by identifying devices and operating systems on a network using passive and active scanning techniques that helped us to understand what systems were present and how they communicate. The lab then instructed us how to gather more detailed information about network hosts and open ports which showed how investigators ultimately discover potential entry points or suspicious activity.

The next portion we worked on was memory-related analysis and this is where we used tools to search for sensitive data such as usernames, passwords, and browsing history that may remain in the system swap space. The lab also showed us how credentials can sometimes be recovered directly from memory, which is a huge security risk. Additional tools were used to check for hidden threats like rootkits and unusual system behavior. The last topic the lab introduced was disk image analysis which allows investigators to examine files and system structures without altering the original data. Overall, the lab highlighted practical techniques for uncovering digital evidence and understanding system activity.

Part 2: Technical summary

In this lab, we continued to experiment with Linux Forensics. We concentrated on analyzing artifacts and began by identifying devices and operating systems with **p0f**. To install **p0f**, we inserted the command **sudo apt-get install p0f**. **P0f** uses a fingerprinting technique based on passively analyzing the structure of a TCP/IP packet to determine the operating system and other configuration properties of a remote host. **P0f** has many options available, including listening to a specific device, writing results to a log file, switching the card to promiscuous mode, listing all available interface, and more. The default fingerprint database of **p0f** is stored in **/etc/p0f/p0f.fp**. Using the **cat** command, we will display the signatures of various packets/protocols.

```
(kali@kali)-[~]
$ cat /etc/p0f/p0f.fp
;
; p0f - fingerprint database
;
;
; See section 5 in the README for a detailed discussion of the format used here.
;
; Copyright (C) 2012 by Michal Zalewski <lcamtuf@coredump.cx>
;
; Distributed under the terms and conditions of GNU LGPL.
;
classes = win,unix,other
;
; =====
; MTU signatures
; =====

[mtu]

; The most common values, used by Ethernet-homed systems, PPP over POTS, PPPoE, DSL, etc:
label = Ethernet or modem
```

```
label = Ethernet or modem
sig  = 576
sig  = 1500

; Common DSL-specific values (1492 is canonical for PPPoE, but ISPs tend to
; horse around a bit):

label = DSL
sig  = 1452
sig  = 1454
sig  = 1492

; Miscellaneous tunnels (including VPNs, IPv6 tunneling, etc):

label = GIF
sig  = 1240
sig  = 1280

label = generic tunnel or VPN
sig  = 1300
sig  = 1400
sig  = 1420
sig  = 1440
sig  = 1450
sig  = 1460
```

Since the output was extremely lengthy, I only included the first chunk of it in the screenshots above.

To display the available interfaces, we entered the command **p0f -L**.

```
(kali@kali)-[~]
$ p0f -L
-- p0f 3.09b by Michal Zalewski <lcamtuf@coredump.cx> --

-- Available interfaces --

0: Name      : eth0
   Description: -
   IP address : 192.168.0.148

1: Name      : any
   Description: Pseudo-device that captures on all interfaces
   IP address : (none)

2: Name      : lo
   Description: -
   IP address : 127.0.0.1

3: Name      : bluetooth-monitor
   Description: Bluetooth Linux Monitor
   IP address : (none)

4: Name      : nflog
   Description: Linux netfilter log (NFLOG) interface
   IP address : (none)
```

```

5: Name      : nfqueue
   Description : Linux netfilter queue (NFQUEUE) interface
   IP address  : (none)

6: Name      : dbus-system
   Description : D-Bus system bus
   IP address  : (none)

7: Name      : dbus-session
   Description : D-Bus session bus
   IP address  : (none)

```

This output showed seven different interfaces. The list included the name, description and IP address of every interface. When using the **p0f** command without any options, this starts the fingerprint process on the local machine. We entered the command **sudo p0f** to begin this process.

```

(kali@kali)-[~]
$ sudo p0f
— p0f 3.09b by Michal Zalewski <lcantuf@coredump.cx> —

[+] Closed 1 file descriptor.
[+] Loaded 322 signatures from '/etc/p0f/p0f.fp'.
[+] Intercepting traffic on default interface 'eth0'.
[+] Default packet filtering configured [+VLAN].
[+] Entered main event loop.

.-[ 192.168.0.148/51976 → 95.17.81.188/80 (syn) ]-
|
| client   = 192.168.0.148/51976
| os       = Linux 2.2.x-3.x
| dist     = 0
| params   = generic
| raw_sig  = 4:64+0:0:1460:mss+44,9:mss,sok,ts,nop,ws:df,id+:0
|
|
.-[ 192.168.0.148/51976 → 95.17.81.188/80 (mtu) ]-
|
| client   = 192.168.0.148/51976
| link     = Ethernet or modem
| raw_mtu  = 1500
|

```

Since this output was lengthy, I only included the first section of it in the screenshot above.

In the second part of this lab, we experimented with gathering information and fingerprinting with **arp-scan** and **nmap**. **Arp-scan** is used to list the address resolution protocol (ARP) table content in the local network. We used the command **sudo arp-scan 172.27.205.46/24** to display this list.

```

(kali@kali)-[~]
$ sudo arp-scan 172.27.205.46/24
Interface: eth0, type: EN10MB, MAC: 08:00:27:35:16:ea, IPv4: 172.27.205.46
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
WARNING: host part of 172.27.205.46/24 is non-zero
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
172.27.205.6      72:5a:4c:64:b8:15      (Unknown: locally administered)
172.27.205.26     e0:75:26:52:11:70      (Unknown)
172.27.205.0      c2:54:a9:61:49:a6      (Unknown: locally administered)
172.27.205.57     26:1c:e5:2c:8b:91      (Unknown: locally administered)
172.27.205.53     8a:26:d0:68:fa:c0      (Unknown: locally administered)
172.27.205.39     a0:78:17:99:6d:bc      (Unknown)
172.27.205.87     da:79:4d:6c:d6:74      (Unknown: locally administered)
172.27.205.82     ec:8e:b5:05:42:09      (Unknown)
172.27.205.56     c6:63:65:47:37:7c      (Unknown: locally administered)
172.27.205.113    62:42:18:35:30:4a      (Unknown: locally administered)
172.27.205.96     9c:76:13:f3:c8:cf      (Unknown)
172.27.205.96     9c:76:13:f3:c8:cf      (Unknown) (DUP: 2)
172.27.205.156    8c:ea:48:a8:27:5e      (Unknown)
172.27.205.157    cc:ba:97:0a:6f:28      (Unknown)
172.27.205.126    60:ff:9e:9e:f8:fa      (Unknown)
172.27.205.118    9a:27:ba:4d:81:6c      (Unknown: locally administered)
172.27.205.19     b6:75:f3:fd:65:19      (Unknown: locally administered)
172.27.205.173    12:79:9f:54:0e:91      (Unknown: locally administered)
172.27.205.73     9a:cc:92:c1:56:cf      (Unknown: locally administered)

```

```

2)
172.27.205.67 2e:c3:59:e3:5a:1a (Unknown: locally administered)
172.27.205.67 2e:c3:59:e3:5a:1a (Unknown: locally administered) (DUP:
2)
172.27.205.250 a0:85:27:c0:66:f6 (Unknown) (DUP: 5)
172.27.205.134 6c:40:08:8d:b4:7e (Unknown)
172.27.205.250 a0:85:27:c0:66:f6 (Unknown) (DUP: 6)
172.27.205.32 ee:77:93:76:27:1f (Unknown: locally administered)
172.27.205.32 ee:77:93:76:27:1f (Unknown: locally administered) (DUP:
2)
172.27.205.83 8e:9e:19:b5:b5:68 (Unknown: locally administered)
172.27.205.83 8e:9e:19:b5:b5:68 (Unknown: locally administered) (DUP:
2)
172.27.205.250 a0:85:27:c0:66:f6 (Unknown) (DUP: 7)
172.27.205.250 a0:85:27:c0:66:f6 (Unknown) (DUP: 8)
172.27.205.241 16:6f:98:eb:fb:9e (Unknown: locally administered)
172.27.205.241 16:6f:98:eb:fb:9e (Unknown: locally administered) (DUP:
2)
172.27.205.226 2e:20:e5:91:d4:36 (Unknown: locally administered)
172.27.205.226 2e:20:e5:91:d4:36 (Unknown: locally administered) (DUP:
2)
393 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 1.861 seconds (137.56 hosts/sec)
. 42 responded

```

The output was lengthy, so I included the beginning and end of the output. This output lists all the fingerprint IP hosts along with the MAC addresses that matches it on the local network. To go a little bit further, we checked whether the host is alive or not without doing a deep investigation. We performed this task by inputting the command **nmap -sn 172.27.205.46/24**.

```

(kali@kali)-[~]
└─$ nmap -sn 172.27.205.46/24
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-09 19:44 -0500
Nmap scan report for 172.27.205.0
Host is up (0.020s latency).
MAC Address: C2:54:A9:61:49:A6 (Unknown)
Nmap scan report for 172.27.205.6
Host is up (0.0046s latency).
MAC Address: 72:5A:4C:64:B8:15 (Unknown)
Nmap scan report for 172.27.205.10
Host is up (0.78s latency).
MAC Address: BE:50:1D:98:DC:03 (Unknown)
Nmap scan report for 172.27.205.19
Host is up (0.028s latency).
MAC Address: B6:75:F3:FD:65:19 (Unknown)
Nmap scan report for 172.27.205.26
Host is up (0.011s latency).
MAC Address: E0:75:26:52:11:70 (China Dragon Technology Limited)
Nmap scan report for 172.27.205.32
Host is up (0.021s latency).
MAC Address: EE:77:93:76:27:1F (Unknown)
Nmap scan report for 172.27.205.39
Host is up (0.043s latency).
MAC Address: A0:78:17:99:6D:BC (Apple)
Nmap scan report for 172.27.205.42
Host is up (0.14s latency).
MAC Address: 66:83:60:EF:76:11 (Unknown)

```

```

MAC Address: 94:E7:0B:FE:9F:D9 (Intel Corporate)
Nmap scan report for 172.27.205.228
Host is up (0.044s latency).
MAC Address: 94:F6:D6:14:7D:04 (Apple)
Nmap scan report for 172.27.205.230
Host is up (0.0027s latency).
MAC Address: 00:91:9E:3A:31:8A (Intel Corporate)
Nmap scan report for 172.27.205.231
Host is up (0.0022s latency).
MAC Address: 32:87:91:3B:66:C2 (Unknown)
Nmap scan report for 172.27.205.233
Host is up (0.060s latency).
MAC Address: BA:20:96:34:93:D6 (Unknown)
Nmap scan report for 172.27.205.241
Host is up (0.030s latency).
MAC Address: 16:6F:98:EB:FB:9E (Unknown)
Nmap scan report for 172.27.205.247
Host is up (0.11s latency).
MAC Address: BE:C9:24:FA:B7:8E (Unknown)
Nmap scan report for 172.27.205.250
Host is up.
MAC Address: A0:85:27:C0:66:F6 (Unknown)
Nmap scan report for 172.27.205.46
Host is up.
Nmap done: 256 IP addresses (44 hosts up) scanned in 9.81 seconds

```

This output listed the **nmap** scan report for a specific host, stating whether the host was up or down along with the MAC addresses. I only included the beginning and end of the output because of the length.

To perform a TCP port SYN scan we entered the command **sudo nmap -sS 172.27.205.247**.

```

(kali@kali)-[~]
$ sudo nmap -sS 172.27.205.247
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-09 19:57 -0500
Nmap scan report for 172.27.205.247
Host is up (0.0098s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
5000/tcp  open  upnp
7000/tcp  open  afs3-fileserver
MAC Address: BE:C9:24:FA:B7:8E (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 8.04 seconds

```

This scan displayed the ports that are open along with the service being used and the MAC address. Another port scan we can perform is the UDP port scan. I used the following commands to start the UFW firewall and allow the port 53/udp: **sudo systemctl start ufw**, **sudo ufw allow 53/udp** and **sudo ufw status verbose**.

```

(kali@kali)-[~]
$ sudo systemctl start ufw
[sudo] password for kali:

(kali@kali)-[~]
$ sudo ufw allow 53/udp
Rule added
Rule added (v6)

(kali@kali)-[~]
$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
22 ALLOW IN Anywhere
53/udp ALLOW IN Anywhere
22/tcp (v6) ALLOW IN Anywhere (v6)
22 (v6) ALLOW IN Anywhere (v6)
53/udp (v6) ALLOW IN Anywhere (v6)

```

When checking the UFW status, the list of ports is displayed. This list stated that the ports allow connections in and where exactly they

allow the connections in from. I then opened a second virtual machine and entered the following command: **sudo nmap -sU 172.27.205.247**.

```
(kali㉿kali)-[~]
└─$ sudo nmap -sU 172.27.205.247
[sudo] password for kali:
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-09 20:05 CDT
Nmap scan report for 172.27.205.247
Host is up (0.0053s latency).
Not shown: 999 open|filtered udp ports (no-response)
PORT      STATE SERVICE
137/udp   open  netbios-ns

Nmap done: 1 IP address (1 host up) scanned in 5.06 seconds
```

While running this scan, the second virtual machine listed the port that was open along with the service it uses.

In the third part of this lab, we focused on gathering information using **swap_digger**. **Swap_digger** is used to perform analysis of the Linux swap file to retrieve system passwords, usernames, credentials, and other important information. To begin using this tool, we need to change the directory using **cd /home/kali** and then input **git clone https://github.com/sevagas/swap_digger.git**.

```
(kali㉿kali)-[~]
└─$ git clone https://github.com/sevagas/swap_digger.git
Cloning into 'swap_digger' ...
remote: Enumerating objects: 147, done.
remote: Counting objects: 100% (30/30), done.
remote: Compressing objects: 100% (19/19), done.
remote: Total 147 (delta 15), reused 21 (delta 11), pack-reused 117 (from 1)
Receiving objects: 100% (147/147), 357.52 KiB | 26.00 KiB/s, done.
Resolving deltas: 100% (69/69), done.
```

After we cloned into **swap_digger**, we switched the directory to **swap_digger** using **cd swap_digger**, input the command **sudo chmod +x swap_digger.sh** as well as **sudo ./swap_digger.sh -S**.

```
(kali㉿kali)-[~]
└─$ cd swap_digger

(kali㉿kali)-[~/swap_digger]
└─$ sudo chmod +x swap_digger.sh
```

```
(kali㉿kali)-[~/swap_digger]
└─$ sudo ./swap_digger.sh -S

- SWAP Digger -

[+] Current swap file:
    → /swapfile
[+] /etc/fstab swap files:
[+] Looking for all available swap device files (will take some time):

SWAP Digger end, byebye!

/home/kali/swap_digger
```

Using the command above, I was able to locate current available swap files. In order to display the data within the **swap_digger** file we use the command **sudo ./swap_digger.sh -a**.

```

(kali@kali)-[~/swap_digger]
$ sudo ./swap_digger.sh -a
[sudo] password for kali:

- SWAP Digger -

[+] Looking for swap partition
    → Found swap at /swapfile
[+] Dumping swap strings in /tmp/swap_dig/swap_dump.txt ... (this may take some time)

=== Web entered passwords and emails ===

[+] Looking for web passwords method 1 (password in GET/POST) ...
[+] Looking for web passwords method 2 (JSON) ...
[+] Looking for web passwords method 3 (HTTP Basic Authentication) ...
[+] Looking for web entered emails ...

=== XML data ===

[+] Looking for xml passwords ...

```

```

=== XML data ===

[+] Looking for xml passwords ...

=== WiFi ===

[+] Looking for wifi access points ...
[-] Potential wifi network list this computer accessed to:

[+] Looking for potential Wifi passwords ...
[-] Potential wifi password list (use them to crack above networks)

[+] Looking for potential Wifi passwords method 2 ...
[-] Potential wifi password list (use them to crack above networks)

=== Mining most accessed resources ===

[+] TOP 30 HTTP/HTTPS URLs (domains only)

[+] TOP 30 FTP URLs

```

```

[+] TOP 30 FTP URLs

[+] TOP 30 .onion urls

[+] TOP 30 files

[+] TOP 30 smb shares

[+] TOP 30 IP addresses (lots of false positives, ex. file versions)

=== Mining hashes ===

[-] No MD5-hashes found
[-] No SHA1-hashes found
[-] No SHA256-hashes found
[-] No SHA512-hashes found
[-] No Blowfish-hashes found

SWAP Digger end, byebye!

/home/kali/swap_digger

```

This output lists many different categories such as web entered passwords and emails, XML data, WiFi, mining resources, and mining hashes. The last **swap_digger** option we examined was **sudo ./swap_digger.sh -p**.

```
(kali@kali)-[~/swap_digger]
$ sudo ./swap_digger.sh -p

- SWAP Digger -

[+] Swap dump already available at /tmp/swap_dig/swap_dump.txt

== Linux system accounts ==

[+] Digging linux accounts credentials... (pattern attack)
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import crypt; print(crypt.crypt("", "$y$j9T"))
ModuleNotFoundError: No module named 'crypt'
Passwords not found. Attempt dictionary based attack? (Can last from 5 minutes to several hours depending on swap usage) [y/n] y

[+] Digging linux accounts credentials method 2 ... (dictionary attack)
[-] Generating wordlist file...
[-] Digging passwords in wordlist... (This may take 5min to few hours!)
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import crypt; print(crypt.crypt("", "$y$j9T"))
ModuleNotFoundError: No module named 'crypt'

SWAP Digger end, goodbye!

/home/kali/swap_digger
```

This command lists passwords from the Linux credential system.

The fourth part of this lab briefly touched on password dumping using **mimipenguin**. To install **mimipenguin** we used the command **git clone** <https://github.com/huntergregal/mimipenguin.git>.

```
(kali@kali)-[~/swap_digger]
$ git clone https://github.com/huntergregal/mimipenguin.git
Cloning into 'mimipenguin'...
remote: Enumerating objects: 536, done.
remote: Counting objects: 100% (7/7), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 536 (delta 3), reused 1 (delta 0), pack-reused 529 (from 2)
Receiving objects: 100% (536/536), 186.58 KiB | 29.00 KiB/s, done.
Resolving deltas: 100% (244/244), done.
```

After successfully installing **mimipenguin**, we switched the directory using **cd mimipenguin** and then entered the command **sudo ./mimipenguin.sh** to run the **mimipenguin** tool to recover user passwords from the system's memory.

```
(kali@kali)-[~/mimipenguin]
$ sudo ./mimipenguin.sh
MimiPenguin Results:
```

This output did not display any results.

In the fifth part of this lab, we dove deeper into Linux digital forensics tools. To check the presence of rootkits, suspicious files, or hidden directories we used the tool **rkhunter**. To install **rkhunter** we used the command **sudo apt-get install rkhunter**. After successfully downloading **rkhunter**, I input the command **sudo rkhunter --check --rwo** to run this tool.


```

(kali@kali)-[~]
└─$ sudo rkhunter --check --rwo
Warning: The file '/usr/bin/mail' exists on the system, but it is not present
in the 'rkhunter.dat' file.
Warning: The command '/usr/bin/lwp-request' has been replaced by a script: /u
sr/bin/lwp-request: Perl script text executable
Warning: The file '/usr/bin/bsd-mailx' exists on the system, but it is not pr
esent in the 'rkhunter.dat' file.
Warning: The following suspicious (large) shared memory segments have been fo
und:
    Process: /usr/bin/thunar      PID: 1452   Owner: kali   Size: 4.0MB
(configured size allowed: 1.0MB)
    Process: /usr/bin/xfdesktop  PID: 1459   Owner: kali   Size: 2.0
MB (configured size allowed: 1.0MB)
    Process: /usr/bin/xfdesktop  PID: 1459   Owner: kali   Size: 32M
B (configured size allowed: 1.0MB)
Warning: The SSH configuration option 'PermitRootLogin' has not been set.
The default value may be 'yes', to allow root access.
Warning: Suspicious file types found in /dev:
/dev/shm/sem.haveged_sem: data
Warning: Hidden directory found: /etc/.java
Warning: Hidden file found: /etc/.updated: ASCII text

```

This command displayed the suspicious files and memory segments within the machine being used. Another tool we can use to check the presence of rootkits is **chkrootkit**. To install this tool, I input **sudo apt-get install chkrootkit**. Once this tool was installed, I entered the command **sudo chkrootkit** to check all the tools that are essential for the computer and display whether or not they are infected.

```

(kali@kali)-[~]
└─$ sudo chkrootkit
ROOTDIR is '/'
Checking 'amd' ... not found
Checking 'basename' ... not infected
Checking 'biff' ... not found
Checking 'chfn' ... not infected
Checking 'chsh' ... not infected
Checking 'cron' ... not infected
Checking 'crontab' ... not infected
Checking 'date' ... not infected
Checking 'du' ... not infected
Checking 'dirname' ... not infected
Checking 'echo' ... not infected
Checking 'egrep' ... not infected
Checking 'env' ... not infected
Checking 'find' ... not infected
Checking 'fingerd' ... not found
Checking 'gpm' ... not found
Checking 'grep' ... not infected
Checking 'hdparm' ... not infected
Checking 'su' ... not infected
Checking 'ifconfig' ... not infected
Checking 'inetd' ... not tested
Checking 'inetdconf' ... not found
Checking 'identd' ... not found
Checking 'init' ... not infected
Checking 'killall' ... not infected

```

This output was extremely long, so I only included the first chunk of information in the screenshot above.

The next tool we focused on was accessing the **ascii** code. To install this tool, I input **sudo apt-get install ascii**. After installing the **ascii** tool, I entered the command **ascii -s hello**.

```

(kali@kali)-[~]
└─$ ascii -s hello
6/5  101  0x65  0o145  1100101
6/8  104  0x68  0o150  1101000
6/12 108  0x6C  0o154  1101100
6/15 111  0x6F  0o157  1101111

```

This command was used to convert each character in the word hello into its **ascii** representation. Another option of the **ascii** tool we looked at was **ascii -x**.

```
(kali@kali)-[~]
$ ascii -x
00 NUL      10 DLE      20      30 0       40 @        50 P        60 `        70 p
01 SOH      11 DC1      21 !       31 1       41 A        51 Q        61 a        71 q
02 STX      12 DC2      22 "       32 2       42 B        52 R        62 b        72 r
03 ETX      13 DC3      23 #       33 3       43 C        53 S        63 c        73 s
04 EOT      14 DC4      24 $       34 4       44 D        54 T        64 d        74 t
05 ENQ      15 NAK      25 %       35 5       45 E        55 U        65 e        75 u
06 ACK      16 SYN      26 &       36 6       46 F        56 V        66 f        76 v
07 BEL      17 ETB      27 '       37 7       47 G        57 W        67 g        77 w
08 BS       18 CAN      28 (       38 8       48 H        58 X        68 h        78 x
09 HT       19 EM       29 )       39 9       49 I        59 Y        69 i        79 y
0A LF       1A SUB      2A *       3A :       4A J        5A Z        6A j        7A z
0B VT       1B ESC      2B +       3B ;       4B K        5B [        6B k        7B {
0C FF       1C FS       2C ,       3C <       4C L        5C \        6C l        7C |
0D CR       1D GS       2D -       3D =       4D M        5D ]        6D m        7D }
0E SO       1E RS       2E .       3E >       4E N        5E ^        6E n        7E ~
0F SI       1F US       2F /       3F ?       4F O        5F _        6F o        7F DEL
```

This command displayed the ascii

table in hexadecimal.

Using the **xxd** tool, we can display a file signature. When inputting the command **xxd -g 1 0zapftis.rar | head** we can list the first ten lines of a .rar file.

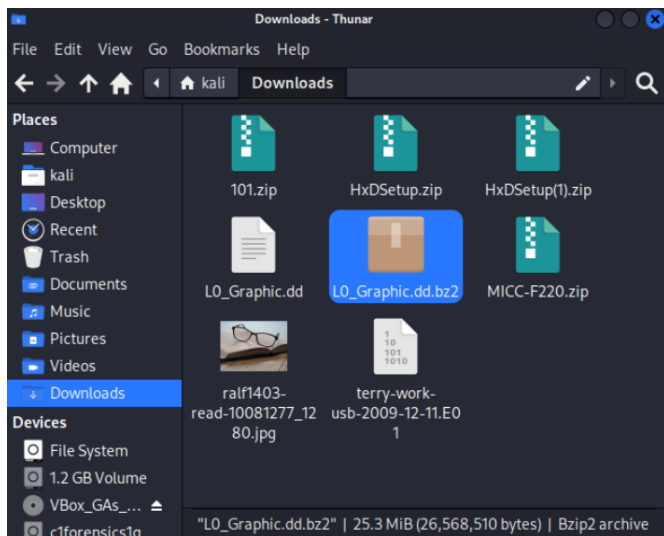
```
(kali@kali)-[~]
$ xxd -g 1 0zapftis.rar | head
00000000: 3c 21 64 6f 63 74 79 70 65 20 68 74 6d 6c 3e 3c <!doctype html><
00000010: 68 74 6d 6c 20 6c 61 6e 67 3d 22 65 6e 22 3e 3c html lang="en"><
00000020: 68 65 61 64 3e 3c 74 69 74 6c 65 3e 45 78 61 6d head><title>Exam
00000030: 70 6c 65 20 44 6f 6d 61 69 6e 3c 2f 74 69 74 6c ple Domain</titl
00000040: 65 3e 3c 6d 65 74 61 20 6e 61 6d 65 3d 22 76 69 e><meta name="vi
00000050: 65 77 70 6f 72 74 22 20 63 6f 6e 74 65 6e 74 3d ewport" content=
00000060: 22 77 69 64 74 68 3d 64 65 76 69 63 65 2d 77 69 "width=device-wi
00000070: 64 74 68 2c 20 69 6e 69 74 69 61 6c 2d 73 63 61 dth, initial-sca
00000080: 6c 65 3d 31 22 3e 3c 73 74 79 6c 65 3e 62 6f 64 le=1"><style>bod
00000090: 79 7b 62 61 63 6b 67 72 6f 75 6e 64 3a 23 65 65 y{background:#ee
```

This output lists the raw

binary data in hexadecimal that makes up this specific file. Another way to check for a specific pattern within a non-text file is to use the command **strings**. When using the command **strings -t x terry-work-usb-2009-12-11.E01 | grep -i "jpg"**, we display the hexadecimal pattern for that jpg file.

```
(kali@kali)-[~/Downloads]
$ strings -t x terry-work-usb-2009-12-11.E01 | grep -i "jpg"
12f44f jPg?0
1217a1c jPg"
13d81b9 LjPg
```

In the last part of the lab, we experimented using the Sleuth toolkit. This kit is a collection of digital forensic tools that can be used to analyze disk images and recover files. I downloaded a .dd file from the website <https://cfreds-archive.nist.gov>.



This file included a **.dd.bz2** file which I had to decompress. In order to list the content of the directory that included these files, I changed the directory to **/home/kali/Downloads** and then entered the command **ls**.

```
(kali@kali) - [~/Downloads]
$ bzip2 -dk L0_Graphic.dd.bz2

(kali@kali) - [~/Downloads]
$

(kali@kali) - [~/Downloads]
$ ls
101.zip          L0_Graphic.dd      ralf1403-read-10081277_1280.jpg
'HxDSetup(1).zip' L0_Graphic.dd.bz2  terry-work-usb-2009-12-11.E01
HxDSetup.zip     MICC-F220.zip
```

To analyze the disk image without mounting it, we can input the command **fls L0_Graphic.dd | head -n 3**.

```
(kali@kali) - [~/Downloads]
$ fls L0_Graphic.dd | head -n 3
Cannot determine file system type
```

Unfortunately, the tool was unable to recognize the **.dd** file and stated it was unable to determine the file system type in the output.

When using the next command, we can display the general details of the file system, specifically the image type we added to the command. I entered **fsstat -i raw L0_Graphic.dd** to do so.

```
(kali@kali) - [~/Downloads]
$ fsstat -i raw L0_Graphic.dd
Cannot determine file system type
```

Once inputting this command, I did not receive the expected output because my device could not determine the file system type. When inputting the command **ils L0_Graphic.dd | head** to attempt to display the inode information in the file system, I received the same output as the previous command.

```
(kali@kali)-[~/Downloads]
$ ls L0_Graphic.dd | head
Cannot determine file system type
```

The second to last command we used to analyze our image file was **img_stat L0_Graphic.dd**. This command allows us to display all the details of an image file.

```
(kali@kali)-[~/Downloads]
$ img_stat L0_Graphic.dd
IMAGE FILE INFORMATION

Image Type: raw

Size in bytes: 64769536
Sector size: 512
```

This output included the image type, the size of the image in bytes as well as the sector size. The last command we examined in this lab was **fiwalk L0_Graphic.dd**. This command is a program that processes a disk image using the Sleuth Kit library and outputs its results in digital forensics XML.

```
(kali@kali)-[~/Downloads]
$ fiwalk L0_Graphic.dd
image_filename: L0_Graphic.dd
fiwalk_version: 4.12.1
start_time: Sun Apr 12 13:59:07 2026
tsk_version: 4.12.1
# TSK_Error 'Cannot determine file system type' at sector 0 offset 0 sector_size=512
# TSK_Error 'Cannot determine file system type' at sector 63 offset 32256 sector_size=512
# TSK_Error 'Cannot determine file system type' at sector 1 offset 512 sector_size=512
# TSK_Error 'Cannot determine file system type' at sector 2 offset 1024 sector_size=512
# TSK_Error 'Cannot determine file system type' at sector 3 offset 1536 sector_size=512
# TSK_Error 'Cannot determine file system type' at sector 4 offset 2048 sector_size=512
# TSK_Error 'Cannot determine file system type' at sector 5 offset 2560 sector_size=512
# TSK_Error 'Cannot determine file system type' at sector 6 offset 3072 sector_size=512
# TSK_Error 'Cannot determine file system type' at sector 7 offset 3584 sector_size=512
# TSK_Error 'Cannot determine file system type' at sector 8 offset 4096 sector_size=512
# TSK_Error 'Cannot determine file system type' at sector 9 offset 4608 sector_size=512
# TSK_Error 'Cannot determine file system type' at sector 57 offset 29184 sector_size=512
# TSK_Error 'Cannot determine file system type' at sector 58 offset 29696 sector_size=512
# TSK_Error 'Cannot determine file system type' at sector 59 offset 30208 sector_size=512
# TSK_Error 'Cannot determine file system type' at sector 60 offset 30720 sector_size=512
# TSK_Error 'Cannot determine file system type' at sector 61 offset 31232 sector_size=512
# TSK_Error 'Cannot determine file system type' at sector 62 offset 31744 sector_size=512
# TSK_Error (do_image) (null)
# clock: 4.949868
utime: 3.816449
stime: 1.137181
maxrss: 17480
minflt: 1838
majflt: 4
nswap: 0
inblock: 1352
oublock: 0
clocktime: 4.949868
# stop_time: Sun Apr 12 13:59:12 2026
# =EOF=
```

Since the output was very lengthy, I only included the first and last parts of the output. Because my device could not determine the file system type of this image file, I received a long list of errors.

Part 3: References

1. Lab 13: Linux Forensics (Artifacts Analysis)

2. [Fsstat command](#)
3. [lfs command](#)
4. [Fiwalk command](#)